

Постановка задачи и требования к решению.

Задача проекта - разработать систему (модель) для автоматического подбора музыкального сопровождения для визуального контента. Система должна по короткому видео и, при наличии, текстовому запросу пользователя выбрать наиболее подходящие музыкальные треки из заранее заданной коллекции.

В отличие от задачи обычной классификации, где для каждого объекта предполагается один правильный класс, подбор музыки является рекомендательной задачей: для одного и того же видео может существовать несколько подходящих треков. Поэтому результатом работы системы должен быть не один заранее определённый ответ, а ранжированный список музыкальных треков-кандидатов с оценками их релевантности.

На вход система получает:

1. короткое видео;
2. название или текстовое описание видео;
3. опциональный текстовый запрос пользователя, например: «спокойная электронная музыка», «рок для динамичного спортивного видео», «грустный инструментальный трек»;
4. коллекцию доступных музыкальных треков.

На выходе система формирует список треков из доступной коллекции, отсортированный по степени соответствия видео и пользовательскому запросу. Для каждого трека может быть рассчитан score совместимости, отражающий вероятность или степень того, насколько данный трек подходит к заданному видео и запросу.

Основные требования к системе описаны в таблице 1.

Таблица 1 – Требования к системе.

Требование	Описание
Семантическая релевантность	Музыка должна соответствовать содержанию видео. Например, для спортивного или динамичного ролика более уместны энергичные треки, а для спокойных сцен — расслабленная или атмосферная музыка.
Эмоциональное соответствие	Система должна учитывать настроение визуального контента и запроса: весёлое, грустное, напряжённое, спокойное, вдохновляющее и т.д.
Учёт пользовательского запроса	Пользователь может явно указать желаемый жанр, настроение, энергию или стиль музыки. Модель должна учитывать этот запрос при ранжировании треков.
Работа с фиксированной коллекцией треков	Система не генерирует музыку и не ищет произвольные треки в интернете, а выбирает один или несколько вариантов из заранее заданного музыкального каталога.
Эффективность вычислений	Желательно использовать небольшие предобученные энкодеры и обучать только лёгкие проекционные и fusion-слои. Это снижает требования к вычислительным ресурсам и потенциально позволяет приблизиться к realtime-инференсу.

Основные ограничения:

1. Субъективность оценки.

Музыкальное сопровождение зависит от вкуса пользователя и контекста использования видео. Поэтому для одного видео трудно определить единственный «правильный» трек. Более корректно рассматривать несколько релевантных вариантов.

2. Ограниченность доступных датасетов.

Открытых датасетов, полностью соответствующих задаче подбора музыки по видео и текстовому запросу, немного. Поэтому разметку и структуру обучающих примеров необходимо формировать самостоятельно на основе доступных видео, аудио и правил совместимости.

3. Лицензионные ограничения.

Музыкальные данные часто защищены авторским правом. Поэтому для экспериментов необходимо использовать треки с подходящей лицензией или открытые датасеты.

4. Многокритериальность соответствия.

Трек может подходить по одному признаку, но не подходить по другому. Например, музыка может совпадать с видео по жанру, но не совпадать по темпу, настроению или уровню интенсивности. Это усложняет разметку и оценку качества.

5. Ограничение фиксированным каталогом.

Система может рекомендовать только те треки, которые есть в доступной коллекции. Если среди них нет подходящего варианта, качество рекомендации будет ограничено составом каталога.

6. Вычислительная сложность мультимодального анализа.

Для решения задачи требуется учитывать текст, видео и аудио. Полное end-to-end обучение таких моделей требует значительных ресурсов, поэтому целесообразно использовать предобученные энкодеры и обучать компактную надстройку поверх заранее извлечённых эмбедингов (например, заранее извлечь эмбединги для всех аудио из коллекции).

Реализация ML-пайплайна

В рамках проекта был реализован полный ML-пайплайн для задачи автоматического подбора музыкального сопровождения к видео с учётом пользовательского текстового запроса. Задача была сформулирована как multilabel-рекомендация: для каждой пары «видео — текстовый запрос» модель должна ранжировать доступные музыкальные треки и присваивать каждому из них оценку релевантности.

Получение данных и разметки

Для проекта был сформирован собственный обучающий датасет пар видео - музыкальный трек на основе двух открытых источников: [OpenLAV](#) и [MTG-Jamendo](#).

В качестве источника видео использовался датасет **Open Library for Affective Videos (OpenLAV)**. Он содержит короткие видеоролики с эмоциональной разметкой. Для каждого видео доступны: идентификатор, название, ссылка на источник, лицензия, длительность и априорная эмоциональная метка. Видео были скачаны по ссылкам из поля `source_URL`, после чего были удалены повторы по названию и обрезаны до длительности, указанной в разметке `length_s`.

Так как в открытом виде отсутствует готовая разметка соответствия между видео из OpenLAV и треками из MTG-Jamendo, обучающие пары были сформированы автоматически. Сначала каждому аудиотреку была назначена одна обобщённая музыкальная категория, например `happy_upbeat`, `calm_relaxing`, `sad_melancholic`, `dark_tense`, `aggressive_intense`, `dreamy_atmospheric` и т.д. Категория определялась на основе жанровых, инструментальных и эмоциональных тегов трека.

Затем была задана матрица совместимости между эмоциональными классами видео и музыкальными категориями. Например, для видео с эмоцией `joy` высокие оценки получали категории `happy_upbeat`, `energetic_electronic` и `playful_fun`, а для `sadness` — `sad_melancholic`, `calm_relaxing` и `dreamy_atmospheric`. На основе этой матрицы для каждой пары `video-track` рассчитывался `compatibility_score` от 0 до 1. Бинарный таргет `target_binary` получался по порогу: если `compatibility_score` ≥ 0.6 , пара считалась подходящей.

В исходной разметке нет ничего связанного с текстовыми запросами пользователя. Поэтому их решено было сгенерировать: для каждого видео было сгенерировано по 40 пользовательских запросов разных типов, например:

- хочу спокойную расслабляющую музыку
- хочу энергичную электронную музыку
- хочу темную напряжённую музыку

На этапе подготовки данных были загружены заранее сформированные train, val и test CSV-файлы. Дополнительное случайное разбиение не выполнялось, чтобы сохранить фиксированный протокол оценки. Исходные данные имели структуру, в которой для каждой пары query_id + video_id перечислялись треки-кандидаты и значение target_binary, показывающее, подходит ли конкретный трек к данному видео и запросу. Для обучения multilabel-модели строки были сгруппированы по query_id + video_id, после чего для каждой группы формировался бинарный вектор длины N, где N — число треков в каталоге. Значение 1 в этом векторе соответствовало релевантному треку, значение 0 — нерелевантному. Также из данных удалялись проблемные видеофайлы, которые не открывались средствами обработки видео.

Архитектура

Основная модель состояла из нескольких блоков. Текстовый блок отдельно принимал эмбединг пользовательского запроса и эмбединг названия видео, после чего объединял их через FFN. Видео-блок проецировал заранее рассчитанный видеоэмбединг в общее пространство признаков. Отдельно использовался числовой признак длительности видео duration_s, который подавался в небольшой блок кодирования длительности. Затем текстовый, визуальный и числовой признаки конкатенировались и проходили через fusion-блок. На выходе модель формировала вектор логитов по всем трекам каталога. Так как задача является multilabel, для обучения использовалась функция потерь BCEWithLogitsLoss.

Получение эмбедингов и обучение

Для ускорения обучения был реализован этап предварительного расчёта эмбедингов. Пользовательские запросы и названия видео кодировались текстовым энкодером intfloat/multilingual-e5-small. Видео кодировались предобученным видеоэнкодером; в итоговой версии для этой цели был выбран VideoMae (MCG-NJU/videomae-base-finetuned-kinetics) как современный видеоэнкодер для извлечения пространственно-временных признаков. Для каждого уникального query_id и video_id эмбединги рассчитывались один раз, сохранялись в .pkl-файлы, а в датафреймы добавлялись индексы query_emb_idx, title_emb_idx и video_emb_idx. Такой подход позволил не пересчитывать тяжёлые признаки на каждой эпохе обучения и существенно ускорил эксперименты.

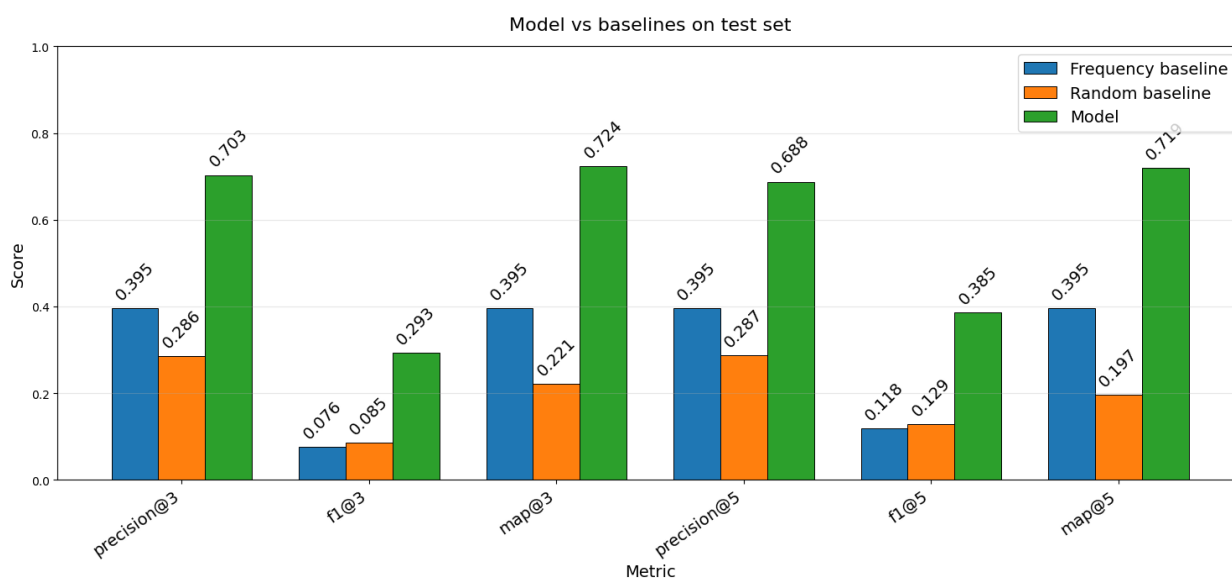
Обучение проводилось батчами с помощью DataLoader. Отдельная collate_fn не требовалась, так как все входные признаки уже имели фиксированные размерности: эмбединги запроса, названия видео, видео и multilabel-вектор. После каждой эпохи модель

оценивалась на валидационной выборке. Для контроля качества использовались ranking-метрики: Precision@K, Recall@K, Hit@K, F1@K и MAP@K. Эти метрики считались для каждой пары «запрос — видео» и затем усреднялись по выборке. Ранний останов выполнялся по валидационной F1@5, так как итоговая задача связана не с калибровкой вероятностей по всем трекам, а с качеством верхней части рекомендательной выдачи.

После обучения лучшая модель сохранялась в checkpoint-файл вместе с историей обучения и значением лучшей метрики.

Кратко метрики

Финальная оценка проводилась на тестовой выборке. Дополнительно результаты сравнивались с двумя бейзлайнами: случайным ранжированием и рекомендацией самых популярных треков



Для анализа результатов были построены графики train/validation loss, графики ranking-метрик по эпохам и диаграммы итоговых test-метрик. Также был проведён анализ ошибок по наилучшим предсказаниям и наихудшим предсказаниям, а также по аудиокатегориям. Такой анализ показал, что модель лучше работает со спокойными и меланхоличными запросами, но хуже справляется с энергичными категориями, что связано с дисбалансом музыкального каталога и обучающей разметки.

В проекте использовались Python, PyTorch для реализации модели и обучения, Transformers для подключения предобученных текстовых и видеоэнкодеров, NumPy и pandas для обработки данных, scikit-learn для вспомогательных метрик, OpenCV для чтения и сэмплирования кадров из видео, tqdm для отображения прогресса и matplotlib для визуализации результатов. Версии библиотек фиксировались в рабочем виртуальном окружении проекта.

Обоснование выбора архитектуры и методов

1. Multilabel-модель без аудиоэмбедингов

Модель получает на вход пользовательский запрос, название видео, само видео и его длительность.

На выходе: вектор из N score-значений, где N — число треков в каталоге

Архитектура состоит из следующих блоков:

Блок	Назначение
Текстовый блок	Кодирует пользовательский запрос и название видео.
Видео-блок	Получает эмбединг видео по нескольким кадрам.
Блок длительности	Кодирует duration s как числовой признак.
Fusion-блок	Объединяет текстовый, визуальный и числовой эмбединги.
Выходной multilabel-блок	Предсказывает релевантность каждого трека из фиксированного каталога.

Текстовый блок нужен, потому что пользовательский запрос и название видео содержат разную информацию. Запрос задаёт желаемый стиль или настроение музыки, а название видео описывает содержание ролика. Их эмбединги (от энкодера «intfloat/multilingual-e5-small») объединяются через FFN, чтобы модель могла учитывать взаимодействие между смыслом видео и предпочтением пользователя.

Видео-блок нужен для учёта визуального содержания: сцены, объектов, динамики, атмосферы и общего визуального стиля. Для этого целесообразно использовать предобученный визуальный энкодер, например CLIP, MobileCLIP или видеоэнкодер вроде R3D-18. В проекте использовались предобученные эмбединги, потому что датасет небольшой, а обучение полноценного видеоэнкодера с нуля привело бы к переобучению и большим вычислительным затратам.

Блок длительности добавлен как вспомогательный признак. Она должна влиять на выбор музыки: коротким роликам чаще подходят треки с быстрым вступлением, а длинным — более устойчивое фоновое сопровождение.

После получения текстового, визуального и числового представлений они конкатенируются и подаются в полносвязную сеть. На выходе модель формирует вектор длины N , где N — число треков в словаре. Так как для одного видео может быть релевантно несколько треков, используется не softmax-классификация, а подходящая Loss функция для multi-label задачи - BCEWithLogitsLoss. Это позволяет модели независимо оценивать релевантность каждого трека.

Преимущество этой архитектуры — простота и скорость. Она хорошо подходит для фиксированного каталога треков и позволяет быстро получать top-K рекомендации. Недостаток — слабая расширяемость: если добавить новый трек, нужно менять размер выходного слоя и дообучать модель.

Обоснование выбора энкодеров:

Текстовый энкодер: `intfloat/multilingual-e5-small`

Для текстового запроса и названия видео использовался `intfloat/multilingual-e5-small`. Выбор связан с тем, что пользовательские запросы могут быть на русском и на английском языке. В [статье](#) 2024 года показывал SOTA результаты, хоть там и сравнение с совсем небольшим числом других моделей проводилось.

Тестировался еще R3D-18 от `torchvision`,

Видеоэнкодер: VideoMamba

В качестве видеоэнкодера выбрана VideoMae, так это современная моделька для извлечения пространственно-временных признаков. По сравнению с R3D-18 VideoMamba является более современной архитектурой, а по сравнению с покадровым CLIP лучше учитывает временную структуру видео. В [статье](#) 2024 года от разработчиков VideoMamba были показаны SOTA метрики почти на всех рассмотренных датасетах при использовании этой модели в качестве бекбона. Размер модели – 87М параметров.

Аудиоэнкодер: `m-a-p/MERT-v1-95M`

Для модели с аудиоэмбедингами был выбран `m-a-p/MERT-v1-95M`. MERT – self-supervised модель для понимания музыки, специально обученная на музыкальном аудио. В отличие от универсальных аудио-моделей, MERT ориентирован на музыкальные признаки: тональность, тембр, структуру, *pitched characteristics* и другие свойства, важные для музыкальных задач. В [статье](#) 2024 года от разработчиков MERT указано, что модель демонстрирует хорошее обобщение на 14 задачах *music understanding* и достигает SOTA overall scores. MERT-v1-95M имеет 95М параметров, 12 слоёв, размерность 768 и sample rate 24 kHz.

Метрики качества и протокол оценки

Для каждого входа может быть сразу несколько подходящих аудиотреков. При этом мы не считаем, что какой-то трек является более подходящим, чем другой (так как нет соответствующей надежной разметки), поэтому метрики DCG / NDCG@K не используем.

Кроме того, в большинстве возможных сценариев использования данной рекомендательной модели пользователь захочет получать сразу несколько возможных рекомендаций, так как выбор музыки очень субъективен.

Также можно предположить, что пользователь не захочет тратить время на перебор более 5 рекомендованных треков и захочет получить релевантные рекомендации в первых 5 или даже 3-х. И желательно чтобы наиболее релевантные треки рекомендовались первыми.

Поэтому в качестве основных метрик использовались Precision@K, Recall@K, Hit@K, F1@K и MAP@K при K = 3, 5.

Precision@K показывает долю релевантных треков среди первых K рекомендаций: если модель выдала K треков, то релевантными будут $K * \text{Precision@K}$ треков. Для данной задачи кажется наиболее важной метрикой, так как она прямо связана с пользовательским опытом использования модели.

Но важно учитывать не только факт предложения релевантного трека, но и его очередность: релевантные треки должны быть выше нерелевантных. Для учета порядка используют **AP@K**.

AP@K: для одного примера входа **AP@K** считается как среднее значение precision предсказаний на позициях, где встречаются релевантные треки.

$$\text{AP@K} = \frac{\sum_{k=1}^K \text{Precision@K} * \text{rel}(k)}{\min(r, K)},$$

где r – количество релевантных треков для данного входа,

K – количество рассматриваемых первых рекомендаций,

rel(k) – индикатор того, что на позиции k стоит релевантный трек.

Затем AP@K усредняют по всем входам, получая **MAP@K**.

Recall@K показывает, какую долю всех релевантных треков модель нашла в первых K рекомендациях: если всего релевантных треков 20, а найдено среди первых пяти – пять, то $\text{Recall@5} = 5/20 = 0.25$.

Эта метрика сильно зависит от количества истинно подходящих аудиотреков для данного входа, поэтому ее низкие значения необязательно значат плохое качество работы модели. В моем случае в среднем для каждого входа подходят 16.25 треков, следовательно, среднее значение Recall@5 в идеальном случае должно быть около $5 / 16.25 = 0.3$.

F1@K — гармоническое среднее Precision@K и Recall@K . Используется для оценки баланса между точностью верхней выдачи и полнотой поиска релевантных треков.

Hit@K показывает долю примеров, где в top-K попал хотя бы один релевантный трек. Метрика хорошо показывает, может ли система предложить пользователю хотя бы один подходящий вариант. Если значения метрики низкие, значит, модель совсем безнадёжна.

Наиболее важными и показательными выделим метрики MAP@K , Precision@K , F1@K .

Оценка качества проводилась на заранее подготовленном разбиении датасета на три части: обучающую, валидационную и тестовую выборки. Разбиение проводилось по входным видео: одинаковые видео не попадали в разные выборки. Пропорции разбиения: 0.7, 0.15, 0.15.

Обучающая выборка использовалась для оптимизации параметров модели, валидационная — для выбора лучшей эпохи и контроля переобучения, тестовая — только для финальной оценки качества.

Перед обучением строки датасета группировались по входным данным. Для каждой группы формировался multilabel-вектор длины, равной числу треков в каталоге: значение 1 ставилось для треков с $\text{target_binary} = 1$, значение 0 — для нерелевантных треков (target_binary – индикатор того, что данное аудио релевантно данному входу).

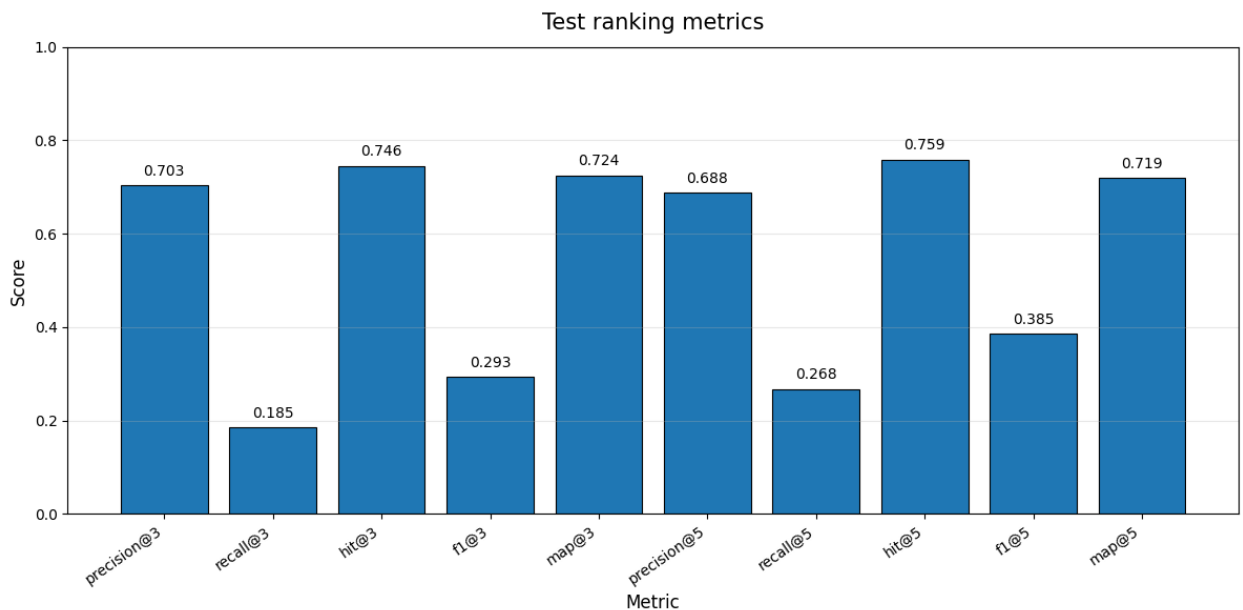
На этапе обучения модель предсказывала score релевантности для каждого трека. Для оптимизации использовалась функция потерь BCEWithLogitsLoss, так как задача является multilabel-задачей: несколько треков могут быть релевантны одновременно.

Валидация проводилась после каждой эпохи. На валидационной выборке рассчитывались Precision@K , Recall@K , Hit@K , F1@K и MAP@K . Эти метрики считались отдельно для каждого входного примера, то есть для каждой пары $\text{query_id} + \text{video_id}$, после чего усреднялись по всей выборке.

Для выбора лучшей модели использовался early stopping по валидационной F1@5 .

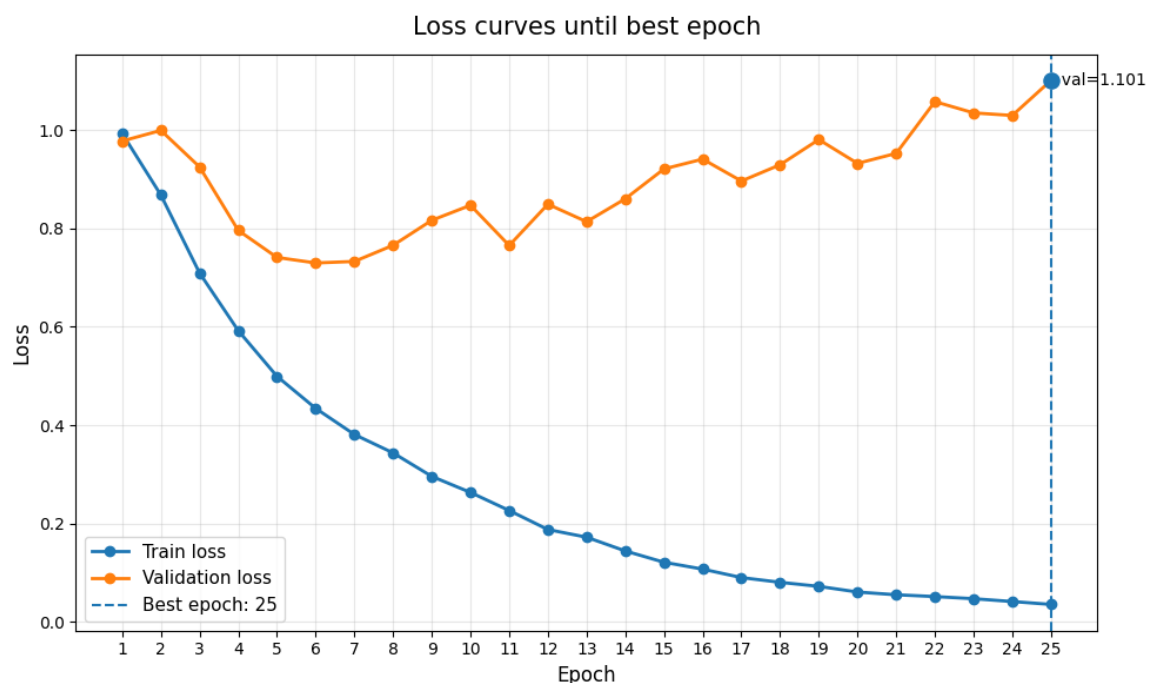
Анализ результатов и интерпретация ошибок модели

График метрик:



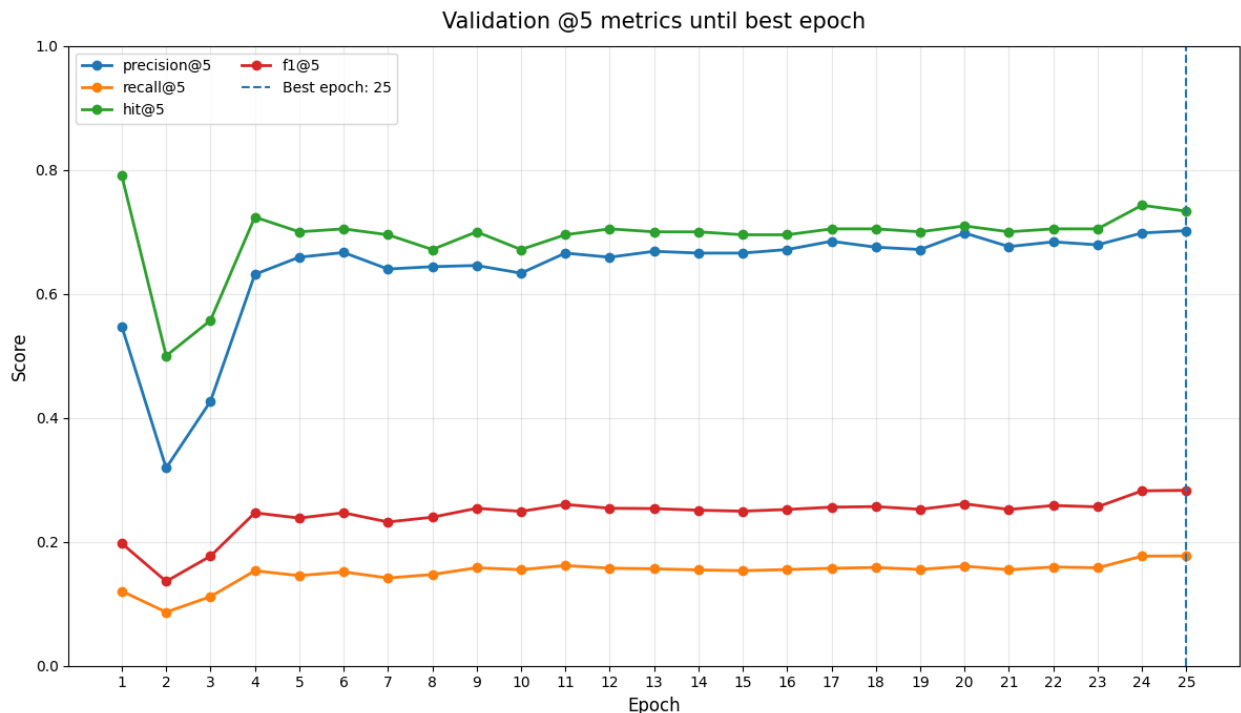
На тестовой выборке модель показывает достаточно сильное качество верхней выдачи: $\text{precision@5} = 0.688$, то есть в среднем около 3.4 из 5 рекомендованных треков релевантны. $\text{MAP@5} = 0.719$ выше precision@5 , что говорит о хорошем порядке внутри top-5: релевантные треки часто оказываются на первых позициях. $\text{recall@5} = 0.268$ выглядит ниже, но это ожидаемо, потому что для одного примера в разметке может быть много релевантных треков, и top-5 физически не может покрыть их все, как это обсуждалось в прошлой главе.

График лоссов по эпохам:



Train loss стабильно убывает почти до нуля, а validation loss после первых эпох начинает расти и к лучшей эпохе достигает 1.101. Это указывает на переобучение и/или ухудшение калибровки вероятностей: модель становится всё более уверенной на обучающей выборке, но не улучшает BCE-loss на валидации. При этом рост validation loss не обязательно означает ухудшение рекомендательного качества, потому что loss оценивает все выходы и их вероятностную калибровку, а основные top-K метрики оценивают порядок треков в верхней части выдачи.

График метрик на валидации по эпохам:



Метрики top-5 после первых нескольких эпох быстро выходят на плато и затем медленно улучшаются. precision@5 держится примерно в диапазоне 0.65–0.70, hit@5 — около 0.70+, а recall@5 остаётся заметно ниже из-за большого числа релевантных треков на один пример. Лучшая эпоха выбрана поздно, потому что ranking-метрики продолжают немного расти, несмотря на рост validation loss; это подтверждает, что для данной задачи ранний останов логичнее делать по top-K метрикам, а не по loss.

Early patience был выбран = 30 эпохам. Дальше метрики точно не росли.

Выделим 3 группы предсказаний по Precision@5:

лучшие случаи: top-5 почти весь релевантный

плохие случаи: top-5 содержит мало или 0 релевантных треков

средние случаи: есть 1–2 релевантных трека, но не идеально

Посмотрим на примеры хороших предсказаний:

row_idx			query_id	video_id	user_query	video_title	num_positive	hits@5	precision@5	recall@5	hit@5
3	3	102_calm_relaxing_tpl_27		102	хочу мягкое спокойное звучание	The Known - Short Horror Film (2014)	36	5	1.0	0.138889	1.0
2	2	102_calm_relaxing_tpl_19		102	подбери легкую фоновую музыку	The Known - Short Horror Film (2014)	36	5	1.0	0.138889	1.0
6	6	102_dreamy_atmospheric_tpl_4		102	подбери музыку с воздушной атмосферой	The Known - Short Horror Film (2014)	36	5	1.0	0.138889	1.0

Для видео 102 и запросов calm_relaxing, dreamy_atmospheric у примеров очень много положительных треков: 36. Поэтому модели проще получить $\text{precision@5} = 1.0$: среди 60 треков много допустимых вариантов, и вероятность попасть в релевантные выше. При этом recall@5 остаётся низким (0.138–0.208), потому что top-5 покрывает только малую часть всех релевантных треков.

Посмотрим на примеры плохого предсказания:

row_idx		query_id	video_id	user_query	video_title	num_positive	hits@5	precision@5	recall@5	hit@5
1	1	102_aggressive_intense_tpl_24	102	хочу интенсивный фон без спокойствия	The Known - Short Horror Film (2014)	1	0	0.0	0.0	0.0
4	4	102_dark_tense_tpl_16	102	подбери dark ambient трек	The Known - Short Horror Film (2014)	4	0	0.0	0.0	0.0
5	5	102_dark_tense_tpl_7	102	подбери драматичный cinematic трек	The Known - Short Horror Film (2014)	4	0	0.0	0.0	0.0

Первый случай – случай с довольно маленьким количеством позитивных классов: $\text{num_positive} = 1$. Если модель даже предлагает эмоционально близкие, но не размеченные как правильные треки, метрика всё равно будет $\text{precision@5} = 0$. Поэтому часть худших кейсов — это не просто “модель ничего не понимает”, а случаи с малым числом допустимых ответов.

Заметим, что видео 102 — The Known - Short Horror Film (2014) — встречается и в лучших, и в худших примерах. Для спокойных/атмосферных запросов модель попадает хорошо, а для dark_tense и aggressive_intense — плохо. Похоже, модель сильно опирается на общий визуальный embedding видео и хуже разделяет близкие или конфликтующие пользовательские намерения. То есть для одного и того же horror-видео она может рекомендовать “атмосферное/спокойное”, но не всегда корректно перестраивается под “напряжённое/агрессивное”, даже если пользователь попросил.

43	43	121_happy_upbeat_tpl_9	121	хочу музыку, которая звучит радостно и тепло	Darkness (Short Horror Film)	21	0	0.0	0.0	0.0
48	48	122_aggressive_intense_tpl_20	122	нужна сильная и напористая музыка	PARANOIA - Short Horror Movie 2015	1	0	0.0	0.0	0.0

Пример 121__happy_upbeat__tpl_9: запрос “радостно и тепло”, но видео называется Darkness (Short Horror Film). Наблюдается противоречие между интенцией пользователя и содержанием видео. Это должно быть нормально в нашей постановке задачи, когда текстовый запрос так же важен, как и само видео, так как пользователи могут специально делать такие контрастные видео. Но модель плохо справилась с таким примером.

Главное подозрение после анализа этих примеров – модель отдает слишком большое предпочтение именно входу в виде видео. Текстовый запрос пользователя и имя видео почти не влияют.

Посмотрим на инференс видео, в котором космонавт выходит в открытый космос на фоне Земли.

video_title	user_query	pred track	genre	audio_category	score
Fighting cats дерутся	Мне нужна напряженная музыка	1313461	chillout	calm_relaxing	0.154
Спокойная безмятежная Земля	Мне нужна спокойная меланхоличная музыка	243844	folk	calm_relaxing	0.999
Космонавт в опасности терпит крушение	Агрессивная злая музыка тяжелый рок	1059974	classical	calm_relaxing	0.121
Спокойная безмятежная Земля	Агрессивная злая музыка тяжелый рок	1062598	easylisening	romantic_warm	0.104

Всё же модель не выдаёт один и тот же результат только по видео. Текст действительно меняет итоговое ранжирование. Мы поменяли названия видео и/или запрос пользователя и рекомендованный трек тоже поменялся. Это радует.

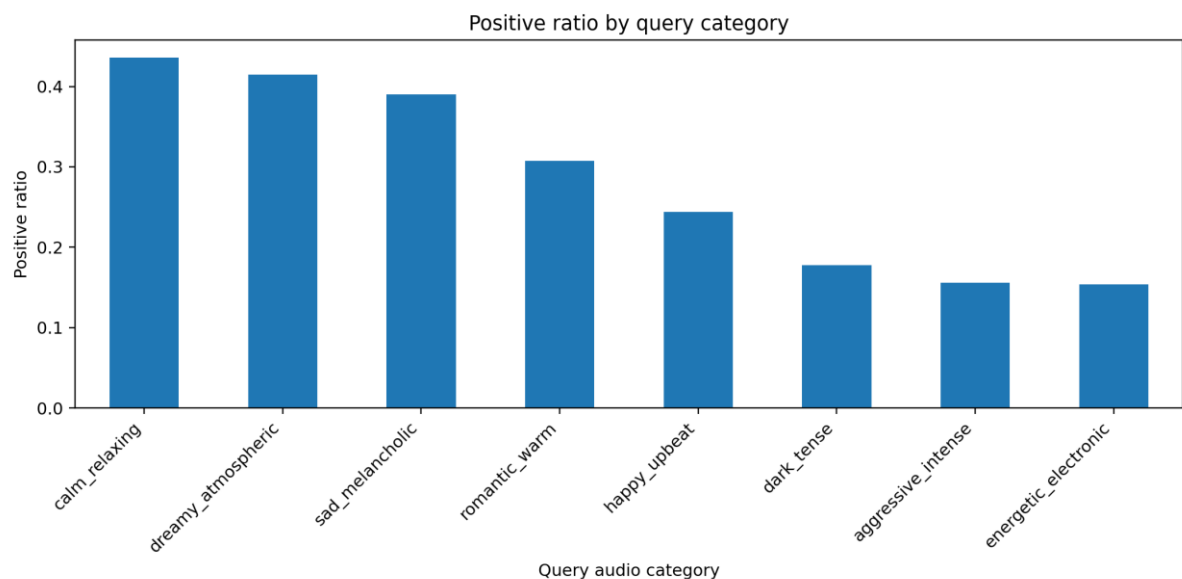
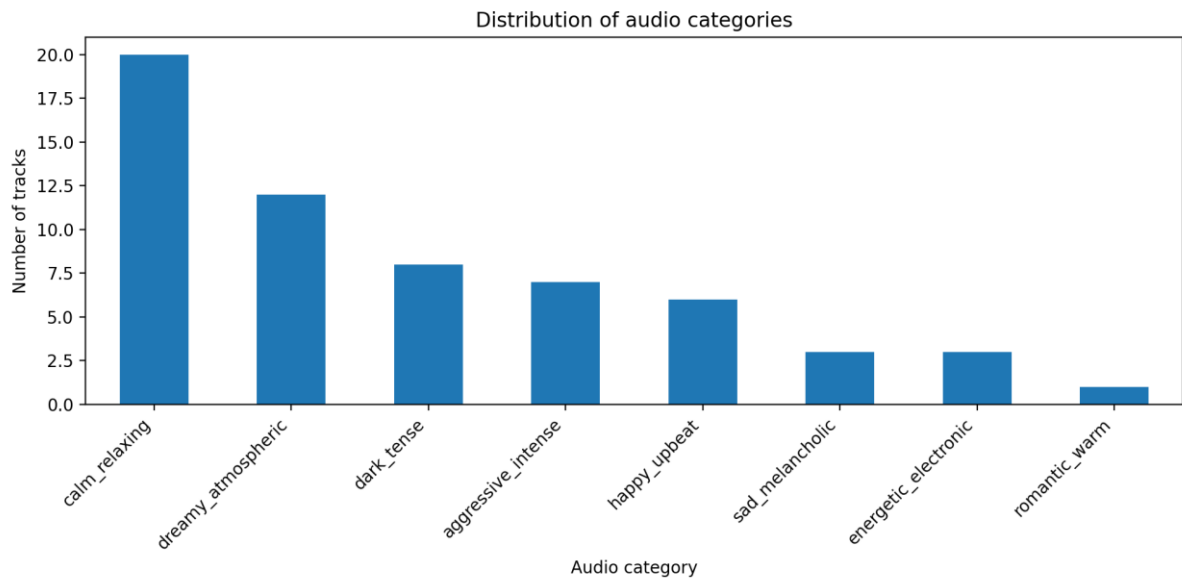
Заметим, что наибольший score получил пример, когда video_title и user_query согласованы. А в остальных примерах модель предложила явно не то, что хочет пользователь. Но сделала это с довольно низкой уверенностью, что может нам давать возможность просто отсекать на инференсе неуверенные предсказания и просить пользователя переформулировать запрос.

Еще одно наблюдение – модель в последнем случае порекомендовала аудио из категории romantic_warm, хотя это самая редкая категория в датасете. Возможно, модель не совсем переобучилась на самых частотных классах.

Но на этих примерах модель не смогла предсказать aggressive intence аудио, когда пользователь об этом просил. Возможные объяснения: 1) модель хоть и учитывает

текстовый вход, но принимает решение преимущественно на основе видео; 2) в данных слишком мало aggressive intense / модель переобучилась предсказывать другие классы.

Это могло привести к смещению модели в сторону спокойных и атмосферных рекомендаций. Возможная причина — сочетание дисбаланса каталога и большого числа положительных меток для универсально подходящих спокойных треков.



Посмотрев снова на распределение категорий аудио и Positive ratio, можем заметить, что спокойных категорий намного больше, чем агрессивных и темных. Предположение о переобучении на спокойные песни скорее всего подтверждается.

Ошибки по аудиокатегориям:

Категория	TP в top-5	FP в top-5	FN вне top-5	Precision по категории	Recall по категории	Интерпретация
calm_relaxing	230	144	1570	0.615	0.128	Категория часто попадает в top-5, но даёт много false positive и огромное число false negative. Модель часто предлагает спокойные треки, но при этом не покрывает большую часть релевантных calm-треков.
dreamy_atmospheric	161	72	583	0.691	0.216	Модель достаточно часто корректно выбирает атмосферные треки, но всё ещё пропускает много релевантных вариантов.
dark_tense	191	68	321	0.737	0.373	Одна из наиболее устойчивых категорий: хорошая точность и сравнительно высокий recall.
happy_upbeat	113	61	223	0.649	0.336	Среднее качество: модель частично различает позитивные треки, но есть заметное число FP и FN.
sad_melancholic	33	48	159	0.407	0.172	Слабая категория: модель часто ошибочно предлагает sad-треки и часто пропускает релевантные.
romantic_warm	23	11	71	0.676	0.245	Неплохая точность, но мало примеров; метрики менее устойчивы.
energetic_electronic	0	11	54	0.000	0.000	Модель совсем не находит релевантные energetic/electronic треки, но иногда ошибочно предлагает их.
aggressive_intense	33	1	135	0.971	0.196	Очень высокая точность, но низкий recall: когда модель выбирает aggressive-трек, почти всегда правильно, но выбирает их редко.

Гипотеза о смещении в сторону спокойных категорий частично подтверждается. Видно, что calm_relaxing действительно чаще всего появляется в top-5 и даёт много false positive: 144 FP. Модель склонна переоценивать спокойные треки и иногда предлагает их там, где они не размечены как релевантные.

Однако у calm_relaxing очень много FN. Это связано с тем, что в разметке calm-треки часто являются положительными, их много в каталоге и в ground truth, поэтому top-5 физически не покрывает большую часть таких релевантных треков. Поэтому корректнее говорить о смещении модели и датасета в сторону спокойных/атмосферных категорий.

Особенно проблемная категория — `energetic_electronic`: модель вообще не находит релевантные треки этой категории ($TP = 0$, $Recall = 0$). Это согласуется с ручным анализом, где при запросах на агрессивную или энергичную музыку модель иногда предлагала спокойные варианты. Значит, модель хуже различает активные/энергичные категории, вероятно из-за малого числа таких треков и слабой представленности в обучающих положительных примерах.

Пути доработки модели и данных

1. Расширение и балансировка музыкального каталога

Одна из основных проблем текущей версии — дисбаланс музыкальных категорий. В каталоге преобладают спокойные и атмосферные треки: `calm_relaxing` и `dreamy_atmospheric` составляют значительную часть всех доступных аудиофайлов. Из-за этого модель может смещаться в сторону «безопасных» спокойных рекомендаций и предлагать их даже в случаях, когда пользователь запрашивает агрессивную, энергичную или напряжённую музыку.

Для доработки данных необходимо расширить и сбалансировать музыкальный каталог: добавить больше треков в недостаточно представленные категории, например `energetic_electronic`, `sad_melancholic`, `romantic_warm`, `aggressive_intense`, `happy_upbeat`. Особенно важно увеличить число треков в тех классах, где сейчас мало кандидатов, потому что при малом количестве треков модель редко видит разнообразные положительные примеры и хуже учится различать такие категории.

Также стоит разнообразить жанровый состав: добавить больше `rock`, `electronic`, `orchestral/cinematic`, `ambient`, `hip-hop`, `pop`, `folk`, `classical`, `funk/disco` и других направлений. Это позволит модели лучше различать не только эмоциональные категории, но и жанровые предпочтения пользователя.

2. Расширение и разнообразие видеоданных

Помимо аудиокаталога, неплохо бы расширить и набор видео. Сейчас часть ошибок связана с тем, что модель может слишком сильно опираться на отдельные повторяющиеся типы контента, например `horror`-видео, и хуже обрабатывать случаи, когда пользовательский запрос конфликтует с визуальным контекстом.

3. Переход к модели с аудиоэмбедингами

Текущая `multilabel`-модель без аудиоэмбедингов предсказывает релевантность фиксированных `track_id`. Такой подход хорошо работает для небольшого фиксированного каталога, но имеет важное ограничение: модель фактически учит соответствие между

входом и конкретными идентификаторами треков, а не между видео/запросом и реальным аудиосодержанием.

Поэтому логичным развитием является модель с аудиоэмбедингами. Я ее архитектуру уже написал и реализовал, но она показывала метрики хуже, чем текущая, а времени искать ошибку или объяснять такое поведение нет, тем более что скорее всего ошибка основана в самих данных и их синтетической природе разметки.

Similarity-модель с аудиоэмбедингами

(видео, текстовый запрос, длительность), кандидатный аудиотрек → score совместимости.

Эта модель не предсказывает номер трека напрямую, а сравнивает эмбединг входного контекста с эмбедингом аудиотрека.

Архитектура состоит из следующих блоков:

Блок	Назначение
Текстовый блок	Кодирует пользовательский запрос и название видео.
Видео-блок	Кодирует визуальное содержание ролика.
Блок длительности	Кодирует длительность видео.
Input projection-блок	Объединяет текст, видео и длительность в общий эмбединг входа.
Аудио-блок	Кодирует кандидатный трек.
Audio projection-блок	Проецирует аудиоэмбединг в общее пространство.
Similarity-блок	Считает cosine similarity между входом и аудио.
Score-блок	Преобразует similarity в score совместимости.

В этой архитектуре текст, видео и длительность объединяются в input embedding. Кандидатный аудиотрек отдельно кодируется аудиоэнкодером, например MERT-v1-95M, после чего аудиоэмбединг проецируется в то же пространство, что и input embedding. Далее считается cosine similarity, масштабируется через logit_scale и преобразуется в score через sigmoid.

Эта архитектура учитывает не только ID трека, но и его аудиосодержание: жанр, тембр, настроение, инструментальность и другие музыкальные признаки. Кроме того, она лучше масштабируется: новый трек можно добавить в каталог без изменения выходного слоя модели, достаточно посчитать его аудиоэмбединг.

При этом вычислительная сложность меняется не сильно: аудиоэмбединги необязательно пересчитывать каждый раз: достаточно делать это один раз при добавлении аудиотрека в коллекцию.

4. В разметке можно добавить не только `binary_target`, но и степень релевантности, если есть способ его получения.

Это позволит использовать более информативные метрики, например NDCG@K , и обучать модель не просто отличать релевантные треки от нерелевантных, а ставить лучшие варианты выше частично подходящих.